LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 06 - Inheritance



CUT GEUBRINA DWI MAHARANI

103132400034

S1SD-05-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

PBO menerapkan konsep inheritance/pewarisan untuk menurunkan atribut dan method dari superclass ke subclass guna mengurangi duplikasi kode, yang didukung oleh encapsulation untuk melindungi data menggunakan akses private dan constructor chaining untuk inisialisasi bertingkat. Selain itu, fleksibilitas program ditingkatkan melalui Method Overriding untuk memodifikasi perilaku method di kelas turunan. Method overloading untuk membuat beberapa method bernama sama dengan parameter berbeda, serta Polimorfisme yang memungkinkan array kelas induk menampung berbagai objek turunan secara dinamis.

B. Guided

1. [Guided 1]

```
package guided.gd1;



public class A {

    int x;

    int y;



    void tampilxy(){

        System.out.println("Nilai x = " + x + " " + "Nilai y =
" + y);

    }

}
```

```
package guided.gd1;



public class B extends A{

    int z;

    int hasil;



    void jumlahxyz(){
```

```
        hasil = (x+y+z);


        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

```
package guided.gd1;


public class Demo {
    public static void main(String[] agrs){


        A superClass = new A();
        B subClass = new B();


        superClass.x = 3;
        superClass.y = 4;


        System.out.println("SuperClass: ");
        superClass.tampilxy();


        subClass.x = 1;
        subClass.y = 2;
        subClass.z = 3;


        System.out.println("SubClass: ");
        subClass.jumlahxyz();
    }
```

```
}
```

Output :

```
SuperClass:
Nilai x = 3 Nilai y = 4
SubClass:
Jumlah (x+y+z) = 6
```

2. [Guided 2]

```
package guided.gd2;

public class employee {
    private String nama;
    String departemen;

    public employee(String nama){
        this.nama = nama;
    }

    public void tampilNama(){
        System.out.println("Nama: " + nama);
    }
}
```

```
package guided.gd2;

public class manager extends employee {
  private String alamat;

  public manager(String nama, String alamat){
    super(nama);
    this.alamat = alamat;
  }

  public void tampilInformasi(){
    super.tampilNama();
    System.out.println("Alamat: " + alamat);
```

```
    System.out.println("Departemen: " + super.departemen);
  }
}
```

```
package guided.gd2;

public class Demo {
    public static void main(String[] args) {
        manager adi = new manager("Adi", "Jakarta");
        adi.departemen = "SDM";
        adi.tampilInformasi();
    }
}
```

Output:

```
Nama: Adi
Alamat: Jakarta
Departemen: SDM
```

3. [Guided 3]

```
package guided.gd3;

public class lagu {
    String pencipta;
    String judul;

    void isiParam(String param1){
        judul = param1;
        pencipta = "tidak dikenal";

    }

    void isiParam(String param1, String param2){
        judul = param1;
        pencipta = param2;
    }
```

```
    void tampilkan(){
        System.out.println("judul: " + judul + " " + "pencipta:
" + pencipta);
    }
}
```

```
package guided.gd3;

public class Demo {
    public static void main(String[] args) {
        lagu d,a;
        d = new lagu();
        a = new lagu();

        d.isiParam("Lagu 1");
        a.isiParam("Lagu 2", "aku");

        d.tampilkan();
        a.tampilkan();
    }
}
```

Output:

```
judul: Lagu 1 pencipta: tidak dikenal
judul: Lagu 2 pencipta: aku
```

4. [Guided 4]

```
package guided.gd4;

public class A {
    public void tampilkan(){
        System.out.println("Method milik Class A
dipanggil...");
    }
}
```

```
package guided.gd4;

public class B extends A {
    public void tampilkan(){
```

```
        super.tampilkan();
        System.out.println("Method milik Class B
dipanggil...");
    }

}
```

```
package guided.gd4;

public class Demo {
    public static void main(String[] args) {
        B overidB = new B();
        overidB.tampilkan();
    }
}
```

Output:

```
Method milik Class A dipanggil...
Method milik Class B dipanggil...
```

C. Unguided

Soal Unguided — Modul 6 Inheritance

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa.

Buatlah program Java dengan ketentuan: Penamaan kelas, atribut, dan method ditentukan sendiri oleh Anda dan semua atribut bersifat private.

Ketentuan

1. Kelas induk (model umum)

- Menyimpan **nama model** (teks) dan **jumlah data latih** (bilangan bulat).
- Memiliki *constructor* yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
- Memiliki method untuk menampilkan informasi.

- Memiliki dua method untuk menghitung biaya komputasi dengan nama yang sama (*overloading*):
    - Tanpa parameter: biaya = jumlah data latih × 2.
    - Dengan satu parameter bilangan bulat / jumlah epoch: biaya = jumlah data latih × epoch.

2. Kelas turunan pertama (model regresi)

- Mewarisi kelas induk dan tambahkan atribut **jumlah fitur**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah fitur; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
- Tambahkan **satu method baru** dengan nama yang sama dengan method penghitung biaya, tetapi menerima **dua parameter bilangan bulat** (jumlah *epoch* dan ukuran *batch*): biaya = jumlah data latih × epoch ÷ ukuran batch. Sehingga objek kelas ini memiliki **tiga versi** method penghitung biaya.

3. Kelas turunan kedua (model klasifikasi)

- Mewarisi kelas induk dan menambahkan atribut **jumlah kelas**.
- *Constructor* menerima nama, jumlah data latih, dan jumlah kelas; gunakan *constructor* kelas induk.
- **Override** method: tampilkan informasi dari kelas induk.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
- Versi penghitung biaya dengan parameter *epoch* **tidak** di-override.

4. Kelas turunan ketiga (model klasifikasi *deep learning*)

- Mewarisi **kelas turunan kedua** (bukan kelas induk langsung) dan menambahkan atribut **jumlah layer**.
- *Constructor* menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer; gunakan *constructor* kelas di atasnya.
- **Override** method: tampilkan informasi dari kelas di atasnya.
- **Override** method untuk hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).

5. Class Main

- Buat tiga objek berikut:

Jenis model	Nama	Data latih	Atribut tambahan
Regresi	`"Regresi Linier"`	500	4 fitur
Klasifikasi	`"Decision Tree"`	800	3 kelas
Klasifikasi *deep learning*	`"Neural Network"`	1000	5 kelas, 3 layer

- Simpan ketiga objek tersebut dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Terakhir, tampilkan juga:
    - biaya 10 *epoch* untuk model `"Neural Network"` (ambil dari array),
    - biaya 10 *epoch* untuk model `"Regresi Linier"`,
    - biaya 10 *epoch* dengan ukuran *batch* 50 untuk model `"Regresi Linier"`.

Contoh Output

```
None
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

•

Jawaban :

1. ModelKomputasi.java

```
package unguided;

public class ModelKomputasi {
    private String nama;
    private int jumlahDataLatih;

    public ModelKomputasi(String nama, int jumlahDataLatih){
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih(){
        return jumlahDataLatih;
    }

    public String getNama(){
        return nama;
    }

    public void tampilkanInfo() {
        System.out.println("Model: " + nama + " | Data latih: "
+ jumlahDataLatih);
    }

    public double hitungBiaya(){
        return jumlahDataLatih * 2;
    }

    public double hitungBiaya(int epoch){
        return jumlahDataLatih * epoch;
    }
}
```

2. ModelRegresi.java

```
package unguided;

public class ModelRegresi extends ModelKomputasi{
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int
jumlahFitur){
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    public int getJumlahFitur(){
        return jumlahFitur;
    }

    public void tampilkanInfo(){
        super.tampilkanInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    public double hitungBiaya(){
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    public double hitungBiaya(int epoch, int ukuranBatch){
        return (double) (getJumlahDataLatih() * epoch)
/ukuranBatch;
    }
}
```

3. ModelKlasifikasi.Java

```
package unguided;

public class ModelKlasifikasi extends ModelKomputasi{
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahDataLatih,
int jumlahKelas){
        super(nama, jumlahDataLatih);
```

```
        this.jumlahKelas = jumlahKelas;
    }

    public int getJumlahKelas(){
        return jumalhKelas;
    }

    public void tampilkanInfo(){
        super.tampilkanInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    public double hitungBiaya(){
        return super.hitungBiaya() * jumlahKelas;
    }
}
```

4. DeepLearning.java

```
package unguided;

public class DeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public DeepLearning(String nama, int jumlahDataLatih, int
jumlahKelas, int jumlahLayer){
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    public int getJumlahLayer(){
        return jumlahLayer;
    }

    public void tampilkanInfo(){
        super.tampilkanInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    public double hitungBiaya(){
```

```
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

5. Main.java

```
package unguided;

public class Main {
    public static void main(String[] args){

        ModelKomputasi[] daftarModel = new ModelKomputasi[3];
        daftarModel[0] = new ModelRegresi("Regresi Linier",
500, 4);
        daftarModel[1] = new ModelKlasifikasi("Decision Tree",
800, 3);
        daftarModel[2] = new DeepLearning("Neural Network",
1000, 5, 3);

        double totalBiaya = 0;

        for (ModelKomputasi model : daftarModel) {
            model.tampilkanInfo();
            double biaya = model.hitungBiaya();
            System.out.println("Biaya komputasi: " + (int)
biaya);
            System.out.println("---");
            totalBiaya += biaya;
        }

        System.out.println("Total biaya: " + (int) totalBiaya);

        double biaya10EpochNN = daftarModel[2].hitungBiaya(10);
        System.out.println("Biaya 10 epoch (Neural Network): "
+ (int) biaya10EpochNN);

        double biaya10EpochRegresi =
daftarModel[0].hitungBiaya(10);
```

```
        System.out.println("Biaya 10 epoch (Regresi Linier): "
+ (int) biaya10EpochRegresi);

        double biayaBatchRegresi = ((ModelRegresi)
daftarModel[0]).hitungBiaya(10, 50);
        System.out.println("Biaya 10 epoch batch 50 (Regresi
Linier): " + (int) biayaBatchRegresi);

    }
}
```

Output :

```
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
```

Penjelasan :

Konsep utama yang digunakan adalah inheritance (Pewarisan) yang memungkinkan sebuah kelas mewarisi atribut dan method dari kelas lain. Enkapsulasi digunakan untuk membatasi akses langsung ke data dengan modifier private dan menyediakan melalui constructor dan getter. Super digunakan sebagai kata kunci untuk memanggil constructor dari kelas induknya agar insilisasi tetap lancar. Method overriding membantu

mendefinisikan ulang method yang sudah ada di kelas utama pada kelas warisan agar sesuai dengan kebutuhan kelas tersebut. Methode overloading digunakan untuk membuat method dengan nama yang sama tapi memilki parameter yang berbeda meskipun di kelas yang sama. Berikut beberapa kelas yang diinisialisasikan:

1. ModelKomputasi
   merupakan parent class yang menampung atribut umum yang dimiliki semua model komputasi yang menyimpan variabel nama komputasi dan jumlah data yang dilatih serta menyimpan fungsi tampilkanInfo untuk mencetak nama model dan jumlah data yang dilatih. Serta menggunakan method overloading hitungBiaya untuk mendefinisikan ulang method dengan nama yang sama tapi memiliki parameter yang berbeda

2. ModelRegresi
   Mewarisi parent class dan menambahkan atribut khusus yaitu jumlah fitur model dan mengubah method yang sudah ada di parent class untuk menyesuaikan dengan kebtuhan pada class ini dengan method Overriding. Method overloading juga digunakan pada hitung biaya dengan menambahkan ukuran batch

3. ModelKlasifikasi
   Model ini mewarisi dengan menambahkan atribut jumlahKelas. Melakukan override pada fungsi tampilkanInfo dan hitungBiaya untuk menyesuaikan kebutuhan kelas ModelKlasifikasi itu sendiri

4. DeepLearning
   Kelas ini melakukan Multilevel Inheritance yaitu mewarisi kelas ModelKlasifikasi. Atribut tambahannya jumlahLayer. Dan menggunakan fitur override di tampilkanInfo dan hitungBiaya untuk menyesuaikans sesuai kebutuhan kelas

5. Main
   Befungsi untuk menjalankan program, membuat objek, memanfaatkan array, dan mencetak hasil ke layar

D. Kesimpulan
   Berdasarkan praktikum mengenai Inheritance (Pewarisasian) ini, dapat disimpulan bahwa penerapan konsep seperti pewarisan kelas, enkapsulasi, constructor chaining, method overriding, method overloading, serta polimorfisme memungkinkan pembuatan program yang terstruktur, modular, dan reuseable. Program dapat mengelola berbagai kelas secara fleksibel dan menyimpan model machine learning dalam satu struktur data array,

sekaligus menyesuaikan perhitungan biaya komputasi dan informasi spesifik setiap model secara dinamis dan akurat